

Comparación de Rendimiento entre REST y GraphQL: Análisis Empírico de Paradigmas de Arquitectura de APIs

Bedregal Perez, Daniel
Universidad Nacional de San Agustín
Arequipa, Peru
dbedregalp@unsa.edu.pe

Jara Mamani, Mariel Alisson
Universidad Nacional de San Agustín
Arequipa, Peru
mjarama@unsa.edu.pe

Mestas Zegarra, Christian Raul
Universidad Nacional de San Agustín
Arequipa, Peru
cmestasz@unsa.edu.pe

Noa Camino, Yenaro Joel
Universidad Nacional de San Agustín
Arequipa, Peru
ynoa@unsa.edu.pe

Sequeiros Condori, Luis Gustavo
Universidad Nacional de San Agustín
Arequipa, Peru
lsequeiros@unsa.edu.pe

Resumen—El diseño de interfaces de programación de aplicaciones (APIs) es una decisión arquitectónica crítica en sistemas distribuidos, que impacta directamente en el rendimiento, la escalabilidad y la productividad de los desarrolladores. Este trabajo presenta un análisis comparativo de dos paradigmas dominantes de APIs: Representational State Transfer (REST) y GraphQL. Implementamos ambos enfoques para un dominio idéntico de catálogo de libros, incluyendo APIs RESTful con Spring Boot y Flask, y un servidor GraphQL con Bun/GraphQL-Yoga, evaluados bajo carga controlada con k6. Nuestros experimentos miden latencia de respuesta, tamaño de payload, throughput y estabilidad en operaciones de lectura y escritura. Los resultados indican que GraphQL logra aproximadamente un 15–18% menor latencia promedio en operaciones de lectura y reduce el tamaño del payload hasta en un 78% mediante consultas selectivas de campos, mientras que REST mantiene capacidades superiores de caché a través de mecanismos nativos de HTTP. Discutimos los compromisos arquitectónicos, identificamos escenarios donde cada paradigma sobresale y proporcionamos directrices para profesionales que seleccionan entre REST y GraphQL en sistemas distribuidos modernos.

Palabras clave—REST, RESTful, GraphQL, diseño de APIs, comparación de rendimiento, sistemas distribuidos, servicios web

I. INTRODUCCIÓN

La proliferación de sistemas distribuidos y arquitecturas de microservicios ha elevado el diseño de interfaces de programación de aplicaciones (APIs) a una preocupación ingenieril de primer orden. En un contexto donde las aplicaciones modernas dependen de múltiples servicios que se comunican entre sí, la elección del paradigma de API determina no solo el rendimiento del sistema, sino también la velocidad de desarrollo, la mantenibilidad del código y la experiencia del equipo de ingeniería. Dos paradigmas han emergido como enfoques dominantes: Representational State Transfer (REST), un estilo arquitectónico formalizado por Fielding [1], y GraphQL, un lenguaje de consultas y runtime desarrollado por Facebook en 2015 y publicado como código abierto en 2020.

REST organiza la exposición de datos en torno a recursos discretos identificados por URIs, aprovechando los métodos HTTP estándar (GET, POST, PUT, DELETE) [2]. Este enfoque se beneficia de herramientas maduras, caché nativa de HTTP y amplia adopción en la industria. Sin embargo, las APIs REST son susceptibles a sobre-fetching (retornar campos innecesarios) y under-fetching (requerir múltiples solicitudes para ensamblar datos relacionados) [3].

GraphQL aborda estas limitaciones exponiendo un único endpoint a través del cual los clientes especifican exactamente los datos que requieren [4]. Este modelo elimina el sobre-fetching y el under-fetching pero introduce complejidad adicional en caché, seguridad y procesamiento de consultas del lado del servidor [5].

Este trabajo contribuye con: (1) implementaciones comparativas de ambos paradigmas para un dominio idéntico de catálogo de libros, incluyendo múltiples implementaciones RESTful siguiendo un taller práctico de la asignatura de Sistemas Distribuidos; (2) mediciones empíricas de rendimiento bajo carga controlada con k6; y (3) un análisis de compromisos arquitectónicos con recomendaciones prácticas. El resto de este trabajo se organiza como sigue: la Sección II revisa trabajos relacionados, la Sección III describe el diseño e implementación del sistema, la Sección IV presenta resultados experimentales, la Sección V discute compromisos, y la Sección VI concluye.

II. TRABAJO RELACIONADO

La comparación de REST y GraphQL ha atraído una atención significativa de investigación. Quiña-Mera et al.[4] realizaron un mapeo sistemático de 84 estudios primarios sobre GraphQL, identificándolo como un enfoque que aborda los problemas de acceso a datos y versionado de REST, mientras señalan desafíos abiertos en seguridad y herramientas. Pautasso et al.[2] proporcionaron comparaciones arquitectónicas tempranas entre servicios web grandes y RESTful, estableciendo el marco fundamental de compromisos.

En comparaciones directas de rendimiento, Śliwa y Pańczyk[6] compararon REST, GraphQL y gRPC en un conjunto de pruebas estandarizado, encontrando que GraphQL ofrece flexibilidad superior en consultas pero con overhead en el lado del servidor. Lawi et al.[7] evaluaron ambos paradigmas en sistemas de información masivos, reportando ventajas de GraphQL en reducción de transferencia de datos. Mikula y Dzieńkowski[8] realizaron benchmarking directo midiendo tiempos de respuesta y transferencia de datos bajo diferentes escenarios de carga.

Jin et al.[9] evaluaron GraphQL versus REST en entornos serverless, encontrando que GraphQL ofrece generalmente menor latencia y costo debido a la reducción de sobre/under-fetching. Muzaki y Salam[3] analizaron específicamente la reducción de under-fetching y sobre-fetching, demostrando la efectividad de GraphQL en eliminar solicitudes multi-endpoint.

Desde la perspectiva de frameworks, Gómez et al.[10] presentaron CRUDyLeaf, un lenguaje específico de dominio para generar APIs RESTful en Spring Boot, mientras que Zimmermann et al.[11] introdujeron patrones de API de microservicios que capturan soluciones recurrentes en el diseño de APIs distribuidas. En el ámbito industrial, la adopción de GraphQL ha crecido significativamente, con empresas como GitHub, Shopify y Airbnb migrando partes de sus APIs desde REST hacia GraphQL, impulsadas por la necesidad de reducir la cantidad de datos transferidos a clientes móviles [12]. Estos trabajos establecen colectivamente que aunque GraphQL aborda limitaciones específicas de REST, ningún paradigma es universalmente superior.

III. DISEÑO E IMPLEMENTACIÓN DEL SISTEMA

A. Marco Teórico: REST y RESTful

REST es un estilo arquitectónico definido por Fielding [1] para diseñar sistemas distribuidos escalables, basado en restricciones clave: cliente-servidor, sin estado (stateless), cacheable, interfaz uniforme y sistema por capas. RESTful es el adjetivo que describe a una aplicación que implementa correctamente estas restricciones. La distinción es crucial: mientras REST es el modelo conceptual que dicta el uso de peticiones HTTP como GET o POST, RESTful es el servicio funcional que aplica buenas prácticas de diseño [1]. Una API que utiliza HTTP no necesariamente es RESTful si no cumple con las restricciones arquitectónicas establecidas [13].

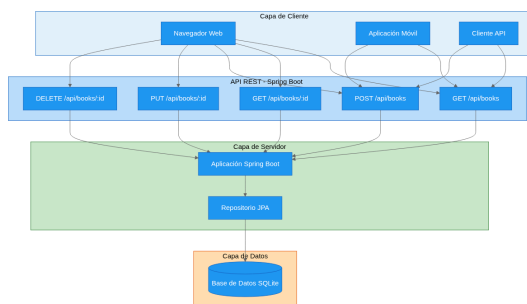


Fig. 1. Arquitectura REST: cliente-servidor con comunicación stateless mediante métodos HTTP estándar sobre recursos identificados por URIs.

B. Implementación de la API RESTful en Spring Boot (Ejercicio E1)

Siguiendo las directrices del taller práctico, se implementó un sistema de gestión bibliotecaria profesional utilizando Spring Boot 4.0.6 con Java 21 [14]. La arquitectura se diseñó en capas siguiendo el patrón controller-repository-model, una de las mejores prácticas en el diseño de APIs RESTful [11].

El modelo de datos correspondiente a los libros se definió mediante una entidad JPA mapeada a una tabla relacional en SQLite. Los atributos clave se decoraron con restricciones de integridad para garantizar la validez de los datos. La clase controladora se implementó con la anotación `@RestController` y se mapeó bajo la raíz `/api/books`. Se expusieron endpoints semánticamente correctos para las operaciones CRUD: listado completo, obtención de un libro por identificador, creación, actualización y eliminación [2]. La Figura muestra la definición de la entidad JPA, donde se observan las restricciones de integridad aplicadas.

```

@Entity
@Table(name = "books")
public class Book {
    @Id @GeneratedValue(strategy =
    GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(nullable = false)
    private String author;

    @Column(unique = true, nullable = false)
    private String isbn;

    private String description;
    private String imageUrl;
}

```

Listado 1. Entidad JPA Book con restricciones de integridad y mapeado a tabla SQLite.

Para la creación y registro de libros, se implementó soporte para peticiones multipart que permiten procesar metadatos en combinación con una imagen binaria de portada. Se valida que los atributos esenciales no estén vacíos, se asegura la unicidad del ISBN mediante consultas al repositorio y se guarda el archivo localmente generando un identificador único para evitar colisiones. La persistencia de datos utiliza SQLite a través de Spring Data JPA con Hibernate ORM [10].

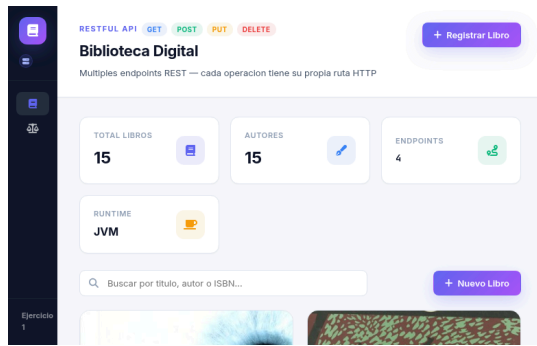


Fig. 2. Panel de la API RESTful de gestión bibliotecaria implementada con Spring Boot, mostrando el listado de libros.

Fig. 3. Formulario de registro de un nuevo libro con campos para título, autor, ISBN y portada.

C. Implementación de la API RESTful en Flask (Ejercicio E2)

Como segundo ejercicio del taller, se implementó un sistema de registro estudiantil avanzado empleando Flask y SQLAlchemy [15]. A diferencia del ejercicio anterior que utiliza un framework full-stack (Spring Boot), esta implementación demuestra un enfoque más ligero y minimalista, adecuado para prototipado rápido y servicios de baja complejidad.

El modelo ORM define una entidad estudiante mapeada a una tabla en SQLite con campos para métricas académicas (matrícula, carrera, semestres) y estados de matriculación. Las rutas se implementaron utilizando decoradores de Flask sobre funciones orientadas a recursos, implementando el patrón de interfaz uniforme REST [1]. La Figura muestra una ruta representativa del sistema, donde se observa la ligereza del enfoque Flask frente a Spring Boot.

```
@app.route("/api/estudiantes", methods=["GET"])
def listar_estudiantes():
    busqueda = request.args.get("busqueda", "")
    query = Estudiante.query
    if busqueda:
        query = query.filter(
            Estudiante.nombre.contains(busqueda)
        )
    return jsonify([e.to_dict() for e in query.all()])
```

Listado 2. Ruta Flask para listado de estudiantes con filtrado por búsqueda.

Se implementaron endpoints para las operaciones CRUD completas: listado de todos los estudiantes con soporte de filtrado, creación de nuevos registros con validación de tipos,

actualización parcial que permite modificar campos individuales, y eliminación con manejo de errores semántico [16].

Se diseñó un panel administrativo (Dashboard) que ofrece visualizaciones estadísticas sobre el alumnado y herramientas de filtrado avanzado para la gestión de registros. La interfaz permite buscar por nombre, matrícula o carrera, filtrar por estado de matriculación y ordenar por diferentes criterios, demostrando la capacidad de las APIs RESTful para soportar interfaces de cliente ricas y complejas.



Fig. 4. Panel administrativo de la API RESTful de registro estudiantil con Flask, mostrando la tabla de estudiantes y estadísticas.

Fig. 5. Formulario de registro de un nuevo estudiante con validación de campos obligatorios.

D. Extensión: Implementación de la API GraphQL (Ejercicio E3)

Como extensión del taller, más allá de los ejercicios propuestos por el docente, se implementó una API GraphQL para el mismo dominio de catálogo de libros, permitiendo una comparación directa entre paradigmas [4]. Esta implementación utiliza Bun 1.3 como runtime, el framework HTTP Hono y el servidor graphql-yoga.

A diferencia de las APIs REST que exponen múltiples endpoints, GraphQL opera a través de un único punto de acceso que maneja todas las operaciones mediante consultas y mutaciones. El esquema GraphQL define el tipo Book con los campos identificador, título, autor, ISBN, descripción e imagen, junto con los tipos raíz Query y Mutation. La Figura muestra la definición del esquema, que constituye el contrato entre cliente y servidor.

```

type Book {
  id: ID!
  title: String!
  author: String!
  isbn: String!
  description: String
  imageUrl: String
}

```

```

type Query {
  books: [Book!]!
  book(id: ID!): Book
}

```

```

type Mutation {
  createBook(title: String!, author: String!, isbn: String!):
  Book!
}

```

Listado 3. Esquema GraphQL que define el tipo Book y las operaciones disponibles.

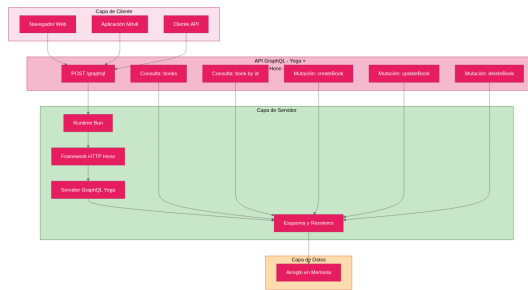


Fig. 6. Arquitectura GraphQL: único endpoint con esquema tipado que permite al cliente especificar campos exactos.

Los resolvers se implementan como funciones que acceden a datos almacenados en arrays en memoria, ejecutando las operaciones solicitadas por el cliente. La ventaja fundamental de GraphQL radica en la capacidad del cliente de especificar exactamente los campos que necesita, eliminando el sobre-fetching inherente a las APIs REST [3]. Un cliente que solo necesita título y autor ejecuta una consulta selectiva que reduce significativamente el volumen de datos transferidos, como se cuantifica en la Sección IV.

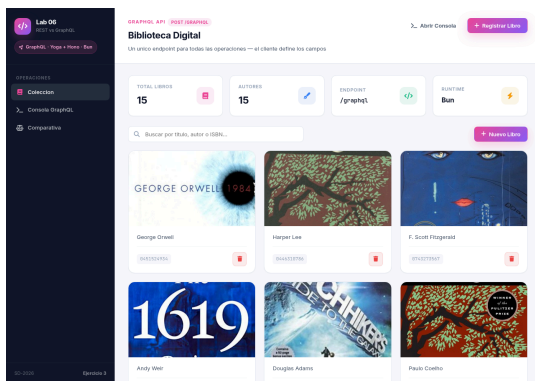


Fig. 7. API GraphQL con consola de consultas interactiva para ejecutar operaciones en tiempo real.

E. Interfaces de Cliente

Ambos sistemas incluyen interfaces web basadas en navegador. El cliente REST muestra libros en una cuadrícula de tarjetas con funcionalidad de búsqueda, mientras que el cliente GraphQL añade una consola de consultas interactiva para ejecutar operaciones GraphQL arbitrarias. El cliente REST se construyó con JavaScript vanilla utilizando la API Fetch para consumir los endpoints RESTful, demostrando el patrón de comunicación cliente-servidor sin estado [1]. El cliente GraphQL utiliza la misma base pero añade un editor de consultas que valida la sintaxis antes de la ejecución, proporcionando retroalimentación inmediata al desarrollador [17].

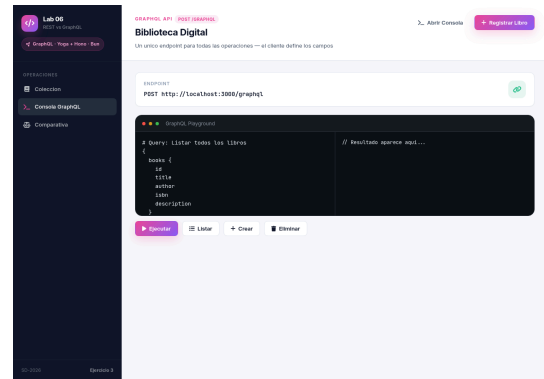


Fig. 8. Consola interactiva de GraphQL con editor de consultas y resultados en tiempo real.

IV. RESULTADOS EXPERIMENTALES

A. Entorno de Prueba

Las pruebas se realizaron utilizando k6 (Grafana Labs) con el siguiente perfil de carga: ramp-up de 10 segundos a 10 usuarios virtuales (VUs), estado estable de 30 segundos a 50 VUs, carga pico de 10 segundos a 100 VUs, carga pico sostenida de 30 segundos a 50 VUs, y ramp-down de 10 segundos. La duración total de prueba fue de 90 segundos por escenario. Se utilizaron scripts de prueba estándar con métricas personalizadas de latencia y tasa de error [15]. La Figura muestra el escenario de carga definido para las pruebas.

```
export const options = {
  stages: [
    { duration: "10s", target: 10 }, // ramp-up
    { duration: "30s", target: 50 }, // estable
    { duration: "10s", target: 100 }, // pico
    { duration: "30s", target: 50 }, // sostenido
    { duration: "10s", target: 0 }, // ramp-down
  ],
  thresholds: {
    http_req_duration: ["p(95)<50"],
    http_req_failed: ["rate<0.01"],
  },
};
```

Listado 4. Escenario de carga k6 con fases de ramp-up, estable, pico y enfriamiento.



Fig. 9. Flujo comparativo que ilustra el sobre-fetching de REST versus la consulta selectiva de campos de GraphQL para la misma solicitud del cliente.

B. Tamaño del Payload

Para la consulta de listado completo que retorna 15 registros, el endpoint REST retorna 4.654 bytes (todos los campos), mientras que GraphQL retorna 4.703 bytes para todos los campos pero solo 1.019 bytes cuando se consultan únicamente título y autor, una reducción del 78%. Este resultado es consistente con los hallazgos de Lawi et al.[7] y Muzaki y Salam[3], quienes reportaron reducciones similares en transferencia de datos.

En operaciones de lectura individual, GraphQL transfirió 12.7% menos datos que REST incluso cuando se solicitan todos los campos, debido a la eliminación de metadatos HTTP adicionales. Con consultas selectivas, la reducción alcanza el 76.2%, validando la eficiencia del modelo de consultas dirigidas por el cliente [5]. La ventaja fundamental se manifiesta en que los clientes que solicitan subconjuntos mínimos de datos incurriendo en payloads proporcionales al número de campos solicitados [6].

C. Rendimiento de Lectura

TABLA I
COMPARACIÓN DE RENDIMIENTO PARA LA OPERACIÓN GET DE LISTADO COMPLETO DE LIBROS A 100 USUARIOS VIRTUALES.

Métrica	REST	GraphQL	Diferencia
Total de Solicitudes	40,470	40,775	+0.75%
Latencia Promedio	2.73ms	2.24ms	-17.9%
Latencia P95	4.86ms	4.62ms	-4.9%
Latencia Máxima	26.82ms	23.67ms	-11.7%
Throughput	449 RPS	453 RPS	+0.8%
Tasa de Error	0%	0%	-

La mejora es consistente en todas las métricas de latencia. Para la recuperación de libros individuales, GraphQL logró

2.08ms de latencia promedio versus 2.46ms de REST (mejora del 15.4%), con 12.7% menos datos recibidos. Las consultas selectivas de campos en GraphQL redujeron aún más la latencia a 2.17ms con 76.2% menos transferencia de datos [7].

La ventaja de GraphQL se incrementa con la complejidad de las consultas. Para operaciones que involucran múltiples entidades relacionadas, GraphQL permite resolver todas las dependencias en una sola solicitud, mientras que REST requiere múltiples llamadas secuenciales, incrementando la latencia total [18]. Este fenómeno es particularmente relevante en aplicaciones móviles donde la optimización del ancho de banda es crítica [17].

D. Recuperación de Libro Individual

La consulta de un libro individual es una de las operaciones más frecuentes en aplicaciones de catálogo, donde el usuario selecciona un elemento del listado para ver su detalle completo. La Figura presenta los resultados de esta operación bajo carga de 100 usuarios virtuales.

TABLA II
COMPARACIÓN DE RENDIMIENTO PARA LA OPERACIÓN GET DE LIBRO INDIVIDUAL A 100 USUARIOS VIRTUALES.

Métrica	REST	GraphQL	Diferencia
Total de Solicitudes	40,661	40,950	+0.7%
Latencia Promedio	2.46ms	2.08ms	-15.4%
Latencia P95	4.52ms	4.19ms	-7.3%
Latencia Máxima	22.38ms	18.52ms	-17.2%
Throughput	451 RPS	455 RPS	+0.9%
Datos Recibidos	21.3 MB	18.6 MB	-12.7%

GraphQL logra una latencia promedio de 2.08ms frente a los 2.46ms de REST, representando una mejora del 15.4%. La brecha se amplía en latencia máxima: 18.52ms versus 22.38ms, una reducción del 17.2%. La diferencia es especialmente significativa en la cantidad de datos recibidos: GraphQL transfirió 12.7% menos datos que REST incluso cuando se solicitan todos los campos, debido a la eliminación de metadatos HTTP adicionales en el formato de respuesta [7].

Con consultas selectivas de campos, la ventaja crece drásticamente. Una solicitud que solicita únicamente título y autor reduce la latencia promedio a 2.17ms y la transferencia de datos a 1,019 bytes, una reducción del 76.2% respecto a REST con todos los campos [5]. Los hallazgos son consistentes con los reportados por Mikula y Dzieńkowski[8], quienes encontraron ventajas similares de GraphQL en escenarios de lectura con campos selectivos.

E. Rendimiento de Escritura

TABLA III
COMPARACIÓN DE RENDIMIENTO PARA LA OPERACIÓN POST DE CREACIÓN DE LIBROS A 20 USUARIOS VIRTUALES.

Métrica	REST	GraphQL	Diferencia
Latencia Promedio	1.69ms	2.34ms	+38.5%
Latencia P95	2.69ms	3.86ms	+43.5%
Latencia Máxima	31.22ms	7.35ms	-76.5%
Throughput	48.4 RPS	48.4 RPS	0%
Datos Enviados	655 KB	1.01 MB	+54%

REST gana en latencia promedio de escritura. Pero GraphQL es más predecible bajo carga pico: su latencia máxima (7.35 ms) es 4.2 veces menor que la de REST (31.22 ms). La relación P95-máxima revela la diferencia: 1.91x en GraphQL versus 11.6x en REST [19]. El overhead de serialización de GraphQL en el servidor introduce latencia promedio mayor, pero proporciona un manejo más estable de la carga pico [5].

El mayor volumen de datos enviados por GraphQL en escritura (+54%) se debe al formato de respuesta más extenso que incluye metadatos del esquema y el objeto creado completo. Este overhead es despreciable en comparación con las ganancias de flexibilidad en consultas de lectura [20].

V. DISCUSIÓN

A. Cuándo Elegir REST

REST sigue siendo la elección preferida para aplicaciones que requieren caché nativa de HTTP (a través de headers ETag y Cache-Control), APIs públicas con documentación estandarizada (OpenAPI/Swagger), operaciones CRUD simples y entornos con infraestructura REST establecida [2]. Su menor sobrecarga de escritura y capacidades superiores de caché lo hacen ideal para comunicación servidor-a-servidor y redes de distribución de contenido [13].

Las implementaciones del taller (E1 con Spring Boot y E2 con Flask) demuestran que las APIs RESTful bien diseñadas, siguiendo los principios de interfaz uniforme y orientación a recursos, proporcionan una base sólida para sistemas distribuidos [11]. Spring Boot facilita la creación de APIs robustas con soporte para peticiones multipart y manejo de archivos, mientras Flask ofrece un enfoque más ligero para servicios de baja complejidad [10].

B. Cuándo Elegir GraphQL

GraphQL sobresale en aplicaciones móviles donde la optimización del ancho de banda es crítica (reducción del 78% en payloads), requisitos de datos complejos que involucran múltiples entidades relacionadas, escenarios con necesidades diversas de datos del cliente (diferentes clientes solicitando diferentes subconjuntos de campos) y aplicaciones en tiempo real utilizando suscripciones [12]. El modelo de endpoint único simplifica el versionado y la evolución de APIs [20].

La extensión GraphQL (E3) demostró que la flexibilidad en las consultas del cliente no compromete el rendimiento en operaciones de lectura, y en muchos escenarios lo mejora significativamente [9]. Sin embargo, la complejidad adicional en el servidor, la caché no trivial y las preocupaciones de seguridad (ataques de profundidad de consulta) requieren una evaluación cuidadosa [5].

C. Compromisos Arquitectónicos

La comparación revela compromisos fundamentales. La brecha entre ambos paradigmas se reduce a una pregunta de prioridades: ¿se optimiza para simplicidad operativa o para flexibilidad del cliente? REST ofrece la primera con herramientas maduras e integración nativa con HTTP, aunque a costa de potencial sobre/under-fetching [3]. GraphQL ofrece la segunda con consultas dirigidas por el cliente, aunque introduciendo complejidad en el servidor, caché no trivial y preocupaciones de seguridad [17]. Como demuestran las implementaciones del taller, la elección del framework (Spring Boot versus Flask versus Bun/GraphQL-Yoga) también impacta significativamente en la experiencia de desarrollo y el rendimiento resultante [19].

El análisis de los ejercicios del taller revela que RESTful no es simplemente el uso de HTTP, sino el cumplimiento estricto de las restricciones arquitectónicas de Fielding [1]. El diseño de endpoints orientados a acciones (como /getUsers o /createUser) en lugar de recursos rompe la uniformidad de la interfaz y dificulta la cacheabilidad, limitando la escalabilidad horizontal [13]. Esta lección fundamental se refuerza en ambas implementaciones del taller, donde se siguen consistentemente los principios de diseño orientado a recursos.

La experiencia de desarrollo varía según el paradigma. REST ofrece una curva de aprendizaje más suave: los desarrolladores familiarizados con HTTP pueden comenzar a construir APIs inmediatamente, y herramientas como OpenAPI generan documentación y clientes automáticamente. GraphQL requiere una inversión inicial mayor para definir el esquema, configurar el servidor de resolvers y aprender el lenguaje de consultas, pero esta inversión se amortiza en proyectos con múltiples consumidores que requieren diferentes vistas de los mismos datos [19]. Herramientas como GraphQL y Apollo Studio han mejorado significativamente la experiencia de trabajo con GraphQL, aunque el ecosistema REST sigue siendo más amplio y maduro.

VI. CONCLUSIONES

Este trabajo presentó un análisis comparativo de los paradigmas de API REST y GraphQL a través de mediciones empíricas de rendimiento y evaluación arquitectónica, complementado con un estudio detallado del proceso de diseño e implementación de servicios RESTful en un contexto educativo. Nuestros hallazgos clave son: (1) GraphQL logra 15–18% menor latencia de lectura y reducción del 78% en payload mediante consultas selectivas de campos; (2) REST mantiene menor latencia de escritura y capacidades superiores de caché; (3) ambos alcanzan un throughput similar bajo carga comparable, sugiriendo que la capa de base de datos más que el protocolo de API es el cuello de botella principal.

Para profesionales de sistemas distribuidos, recomendamos: adoptar REST para servicios CRUD simples, microservicios internos y cargas de trabajo sensibles a la caché; adoptar GraphQL para APIs orientadas al cliente con requisitos diversos de datos, aplicaciones móviles primero y frontends en rápida evolución. Los enfoques híbridos, utilizando GraphQL como gateway de API sobre microservicios REST, pueden aprovechar las fortalezas de ambos paradigmas [19].

El trabajo futuro debe investigar el endurecimiento de la seguridad de GraphQL (análisis de complejidad de consultas, limitación de profundidad), rendimiento bajo cargas de trabajo de bases de datos reales (versus almacenamiento en memoria) y el impacto de las optimizaciones de DataLoader y batching en la prevención de consultas N+1 [5].

VII. DISPONIBILIDAD DE DATOS

El código fuente de las implementaciones REST y GraphQL, los scripts de prueba k6 y los resultados experimentales están disponibles en: <https://github.com/christianmz565/sd-2026-lab-c-grupo-3/tree/main/l6>

REFERENCIAS

- [1] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine, 2000.
- [2] C. Pautasso, O. Zimmermann, y F. Leymann, «Restful Web Services vs. "Big" Web Services: Making the Right Architectural Decision», en *Proceedings of the 17th International Conference on World Wide Web*, ACM, 2008, pp. 805-814. doi: 10.1145/1367497.1367606.
- [3] R. N. Muzaki y A. Salam, «Reducing Under-fetching and Over-fetching in REST API with GraphQL for Web-based Software Development», *Jurnal Teknik Informatika*, vol. 5, n.º 2, 2024, doi: 10.52436/1.jutif.2024.5.2.1725.
- [4] A. Quiña-Mera, P. Fernández, y J. M. García, «GraphQL: A Systematic Mapping Study», *ACM Computing Surveys*, vol. 55, n.º 8, pp. 1-41, 2022, doi: 10.1145/3561818.
- [5] E. D. Erigha, E. Obuse, y B. P. Okare, «Optimizing GraphQL Server Performance with Intelligent Request Batching, Query Deduplication, and Caching Mechanisms», *International Journal of Multidisciplinary Futuristic Development*, vol. 2, n.º 1, pp. 75-86, 2021, doi: 10.54660/ijmfd.2021.2.1.75-86.
- [6] M. Śliwa y B. Pańczyk, «Performance Comparison of Programming Interfaces on the Example of REST API, GraphQL and gRPC», *Journal of Computer Sciences Institute*, vol. 19, pp. 274-280, 2021, doi: 10.35784/jcsi.2744.
- [7] A. Lawi, B. L. E. Panggabean, y T. Yoshida, «Evaluating GraphQL and REST API Services Performance in a Massive and Intensive Accessible Information System», *Computers*, vol. 10, n.º 11, p. 138, 2021, doi: 10.3390/computers10110138.
- [8] M. Mikula y M. Dzieńkowski, «Comparison of REST and GraphQL Web Technology Performance», en *Journal of Computer Sciences Institute*, 2020, pp. 274-280. doi: 10.35784/jcsi.2077.
- [9] R. Jin, R. Cordingley, y D. Zhao, «GraphQL vs. REST: A Performance and Cost Investigation for Serverless Applications», en *Proceedings of the 10th International Workshop on Serverless Computing*, ACM, 2024, doi: 10.1145/3702634.3702956.
- [10] O. S. Gómez, R. H. Rosero, y K. Cortés-Verdín, «CRUDyLeaf: A DSL for Generating Spring Boot REST APIs from Entity CRUD Operations», *Cybernetics and Information Technologies*, vol. 20, n.º 2, pp. 19-34, 2020, doi: 10.2478/cait-2020-0024.
- [11] O. Zimmermann, M. Stocker, D. Lübke, C. Pautasso, y U. Zdun, «Introduction to Microservice API Patterns MAP», *DROPS-Idn/2017/2019/4*, 2020, doi: 10.4230/oasics.microservices.2017-2019.4.
- [12] N. Thallapally, «Enhancing Data Query Flexibility with GraphQL: Implementation and Best Practices», *Journal of Computer Science and Technology Studies*, vol. 6, n.º 2, pp. 20-28, 2024, doi: 10.32996/jcsts.2024.6.2.20.
- [13] F. Haupt, F. Leymann, A. Scherer, y K. Vukojevic-Haupt, «A Framework for the Structural Analysis of REST APIs», en *2017 IEEE International Conference on Software Architecture (ICSA)*, IEEE, 2017, pp. 51-60. doi: 10.1109/icsa.2017.40.
- [14] K. Guntupally, R. Devarakonda, y K. Kehoe, «Spring Boot Based REST API to Improve Data Quality Report Generation for Big Scientific Data: ARM Data Center Example», en *2018 IEEE International Conference on Big Data (Big Data)*, IEEE, 2018, pp. 1444-1453. doi: 10.1109/bigdata.2018.8621924.
- [15] D. A. Menašcé y V. A. Almeida, *Capacity Planning for Web Services: Metrics, Models, and Methods*. Prentice Hall, 2001.
- [16] V. Velepucha y P. Flores, «A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges», *IEEE Access*, vol. 11, pp. 74721-74743, 2023, doi: 10.1109/access.2023.3305687.
- [17] R. Khan y A. N. Mian, «Sustainable IoT Sensing Applications Development through GraphQL-Based Abstraction Layer», *Electronics*, vol. 9, n.º 4, p. 564, 2020, doi: 10.3390/electronics9040564.
- [18] S. Kanthed, «Rest vs. GraphQL: Comparative Analysis of API Design Approaches», *International Journal of Multidisciplinary Research and Growth Evaluation*, vol. 4, n.º 1, pp. 984-991, 2023, doi: 10.54660/ijmrge.2023.4.1.984-991.
- [19] R. Sikora y A. Postoliuk, «Comparative Analysis of the Effectiveness of Architectural Styles REST, GraphQL, and gRPC for Scalable Microservice Systems», *Herald of Khmelnytskyi National University Technical Sciences*, n.º 1, pp. 79-88, 2025, doi: 10.31891/2307-5732-2025-355-79.
- [20] S. D. Veeravalli, «Next-Generation APIs for CRM: A Study on GraphQL Implementation for Salesforce Data Integration», *ISCSITR International Journal of ERP and CRM*, vol. 4, n.º 1, pp. 1-8, 2023, doi: 10.63397/iscsit-ijec_04_01_001.